

API Stream

Damian Nicolalde R.

2021

Capturando medios

- La API Stream nos permite acceder a las transmisiones de medios en el dispositivo. Las transmisiones más comunes son las que se realizan con la cámara y el micrófono, pero esta API se ha definido para facilitar acceso a cualquier otra fuente que produce transmisiones de vídeo o audio.
- La API define el objeto **MediaStream** para referenciar las transmisiones de medios y el objeto **LocalMediaStream** para transmisiones generadas por dispositivos locales. Estos objetos tienen una entrada representada por el dispositivo y una salida representada por elementos como **<video>** y **<audio>**, además de otras API.
- Para obtener el objeto **LocalMediaStream** que representa una transmisión, la API incluye un objeto llamado **MediaDevices** que, entre otros, incluye el siguiente método:
getUserMedia(restricciones)—Este método solicita el permiso del usuario para acceder a la transmisión del vídeo o audio y en respuesta genera un objeto **LocalMediaStream** que representa la transmisión. El atributo **restricciones** es un objeto con dos propiedades video y audio que indican el tipo de medio a capturar.

Método `getUserMedia()`

- El método **`getUserMedia()`** realiza una operación asíncrona, lo que significa que intentará acceder a la transmisión mientras se sigue ejecutando el resto del código. Para este propósito, el método devuelve un objeto **Promise** con el que informar del resultado. Este es un objeto específicamente diseñado para controlar operaciones asíncronas. El objeto incluye dos métodos con los que procesa la respuesta.

`then(función)`—Se ejecuta mediante una operación asíncrona en caso de éxito. Si la operación se realiza correctamente, se llama a este método y se ejecuta la función especificada por el atributo.

`catch(función)`—Se ejecuta mediante una operación asíncrona en caso de error. Si se produce un error en la operación, se llama a este método y se ejecuta la función especificada por el atributo.

- Cuando se accede a una transmisión, el método **then()** envía a la función la referencia del objeto **LocalMediaStream** que representa la transmisión. Para mostrarla al usuario, tenemos que asignar este objeto a un elemento **<video>** o **<audio>**. Para este propósito, los objetos **Element** que representan estos elementos incluyen la siguiente propiedad:

srcObject—Esta propiedad declara o devuelve el objeto **MediaStream** que representa la transmisión.

- El dispositivo más común al que podemos acceder para transmitir vídeo es la cámara. El siguiente ejemplo ilustra cómo acceder a la cámara y mostrar la transmisión en la pantalla.

```

1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>API Stream</title>
6      <script>
7          function iniciar() {
8              var promesa = navigator.mediaDevices.getUserMedia({video: true});
9              promesa.then(exito);
10             promesa.catch(mostrarerror);
11
12             function exito(transmision) {
13                 var video = document.getElementById("medio");
14                 video.srcObject = transmision;
15                 video.play();
16             }
17             function mostrarerror(error) {
18                 console.log("Error: " + error.name);
19             }
20         }
21         window.addEventListener("load", iniciar);
22     </script>
23 </head>
24 <body>
25     <section>
26         <video id="medio"></video>
27     </section>
28 </body>
29 </html>

```

- La función **iniciar()** obtiene la transmisión desde la cámara usando el método **getUserMedia()**. Esto genera una solicitud para el usuario. Si el usuario permite a nuestra aplicación acceder a la cámara, se llama al método **then()** del objeto **Promise** y se ejecuta la función **exito()**.
- Esta función recibe el objeto **LocalMediaStream** generado por el método **getUserMedia()** y lo asigna al elemento **<video>** en el documento mediante la propiedad **srcObject**.
- En caso de error, se llama al método **catch()** del objeto **Promise** y se ejecuta la función **mostrarerror()**. La función recibe un objeto con información acerca del error. El error más común es **PermissionDeniedError**, que se genera cuando el usuario deniega acceso al medio o el medio no está disponible por otras razones.
- En este ejemplo no hemos tenido que declarar al atributo **src** del elemento **<video>** debido a que la fuente será la transmisión de vídeo capturada por el código JavaScript, pero podríamos haber declarado los atributos **width** y **height** para cambiar el tamaño del vídeo en la pantalla.

IMPORTANTE

- El método **getUserMedia()** solo se puede ejecutar desde un servidor y el servidor debe ser seguro, lo que significa que la aplicación no funcionará a menos que subamos los archivos a un servidor que utiliza el protocolo https. Si no posee un servidor seguro, puede probar los ejemplos en un servidor local instalado con una aplicación como MAMP

Objeto **MediaStreamTrack**

- Los objetos **MediaStream** (y, por lo tanto, los objetos **LocalMediaStream**) contienen objetos **MediaStreamTrack** que representan cada pista de medios (normalmente una para vídeo y otra para audio). Los objetos **MediaStream** incluyen los siguientes métodos para obtener los objetos **MediaStreamTrack**.

getVideoTracks()—Este método devuelve un array con objetos **MediaStreamTrack** que representan las pistas de vídeo incluidas en la transmisión.

getAudioTracks()—Este método devuelve un array con objetos **MediaStreamTrack** que representan las pistas de audio incluidas en la transmisión.

Propiedades y método de MediaStreamTrack

- Los objetos que devuelven estos métodos incluyen propiedades y métodos para obtener información y controlar la pista de vídeo o audio. Los siguientes son los más usados.

enabled—Esta propiedad devuelve true o false de acuerdo al estado de la pista. Si la pista aún se encuentra asociada a la fuente, el valor es true.

kind—Esta propiedad devuelve el tipo de fuente que representa la pista. Los valores disponibles son audio y video.

label—Esta propiedad devuelve el nombre de la fuente de la pista.

stop()—Este método detiene la pista.

- Si queremos recoger información acerca de la transmisión, tenemos que obtener la transmisión con el método **getUserMedia()** como hemos hecho en el ejemplo anterior, obtener referencias a las pistas con el método **getVideoTracks()** y luego leer los valores de la pista que se devuelve.

```

1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="utf-8">
5   <title>API Stream</title>
6   <style>
7     section {
8       float: left;
9     }
10  </style>
11  <script>
12    function iniciar() {
13      var promesa = navigator.mediaDevices.getUserMedia({video: true});
14      promesa.then(exito);
15      promesa.catch(mostrarerror);
16
17      function exito(transmision) {
18        var video = document.getElementById("video");
19        video.srcObject = transmision;
20        video.play();
21
22        var cajadatos = document.getElementById("cajadatos");
23        var pistasvideo = transmision.getVideoTracks();
24        var pista = pistasvideo[0];
25        var datos = "";
26        datos += "<br> Habilitado: " + pista.enabled;
27        datos += "<br> Tipo: " + pista.kind;
28        datos += "<br> Dispositivo: " + pista.label;
29        cajadatos.innerHTML = datos;
30      }
31      function mostrarerror(error) {
32        console.log("Error: " + error.name);
33      }
34    }
35    window.addEventListener("load", iniciar);
36  </script>
37 </head>
38 <body>
39   <section>
40     <video id="video"></video>
41   </section>
42   <section id="cajadatos"></section>
43 </body>
44 </html>

```

- El método **getVideoTracks()** del objeto **MediaStream** devuelve un array. Debido a que la cámara contiene una sola pista de vídeo, para obtener una referencia a esta pista tenemos que leer el elemento al índice 0 (la primera pista en la lista).
- En el ejemplo, usamos el mismo código del ejemplo anterior para acceder a la transmisión, pero esta vez la pista de la cámara se almacena en la variable *pista*, y luego cada una de sus propiedades se muestran en la pantalla.

IMPORTANTE: las pistas mencionadas aquí son pistas de vídeo o audio. Estos tipos de pistas no tienen nada que ver con las pistas de subtítulos.

```

1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="utf-8">
5   <title>API Stream</title>
6   <script>
7     function iniciar() {
8       var promesa = navigator.mediaDevices.getUserMedia({video: true});
9       promesa.then(exito);
10      promesa.catch(mostrarerror);
11    }
12    function exito(transmision) {
13      var boton = document.getElementById("boton");
14      boton.addEventListener("click", function() { detenertransmision(transmision) });
15      var video = document.getElementById("video");
16      video.srcObject = transmision;
17      video.play();
18    }
19    function mostrarerror(evento) {
20      console.log("Error: " + error.name);
21    }
22    function detenertransmision(transmision) {
23      var pistasvideo = transmision.getVideoTracks();
24      var pista = pistasvideo[0];
25      pista.stop();
26      alert("Transmisión Cancelada");
27    }
28    window.addEventListener("load", iniciar);
29  </script>
30 </head>
31 <body>
32   <section>
33     <video id="video"></video>
34   </section>
35   <nav>
36     <button type="boton" id="boton">Apagar</button>
37   </nav>
38 </body>
39 </html>

```

- Una vez que obtenemos la pista, podemos detenerla con el método **stop()**. En el este ejemplo, agregamos un botón a la interfaz para detener la transmisión cuando lo pulsa el usuario. El listener del evento **click** para el botón solo se agrega si el código es capaz de acceder a la cámara. En caso contrario, el botón no hace nada.
- Además de las propiedades y los métodos introducido anteriormente, los objetos **MediaStreamTrack** también ofrecen eventos para informar del estado de la pista.
 - muted**—Este evento se desencadena cuando la pista no puede facilitar datos.
 - unmuted**—Este evento se desencadena tan pronto como la pista comienza a proveer datos nuevamente.
 - ended**—Este evento se desencadena cuando la pista ya no puede proveer datos. Esto puede deberse a varias razones, desde el usuario que niega el acceso a la transmisión al uso del método **stop()**.